

TXFETCH

Blockchain Transaction Retrieval Tool

Technical Documentation

by xaynov (@ownstates)

1. Introduction

TXFetch is a command-line utility for searching blockchain transactions using indirect evidence. It solves the reverse problem: instead of knowing the transaction hash and looking up its data, the user knows the data and wants to find the hash.

Typical use case: a person knows they sent or received a transfer at approximately a certain time for a certain amount, but the transaction hash was lost or is unknown. TXFetch queries public blockchain APIs, filters the results, and returns a list of candidates with a probability score.

1.1 Supported networks

Network	Coins	API	Time filter	Pagination
TRON	USDT (TRC-20)	TronGrid Events API	min/max_block_timestamp	fingerprint
TON	TON, USDT, NOT	TONAPI / TONCenter v3	start_date / end_date	before_lt / offset
ETH / EVM	USDT, USDC, ETH	Etherscan API	ts → block → tokentx	page / offset
Bitcoin	BTC	mempool.space	block height by time	start_index (25 TX)

1.2 Input parameters

Required:

- Network (TRON, TON, ETH, BTC)
- Coin (USDT, TON, BTC, etc.)
- Transaction time in UTC

Optional:

- Amount with $\pm 1\%$ tolerance by default
- Comment / memo

2. Architecture

The project is organized as a single entry point and four independent adapters, each implementing a unified interface.

```
txfetch/  
├─ main.py           # TUI, parameter input, result output  
├─ requirements.txt  # dependencies
```

```

└─ modules/
    ├── models.py      # TXQuery, TXCandidate, score_candidate
    ├── tron.py        # TRON / USDT-TRC20
    ├── ton.py         # TON / Jetton
    ├── eth.py         # EVM networks
    └── btc.py         # Bitcoin

```

2.1 Data flow

User inputs parameters → TXQuery → adapter queries API → local filtering →
 score_candidate() → TXCandidate list sorted by confidence.

2.2 Key data types

TXQuery

The query object that main.py assembles from user input and passes to the adapter.
 Contains the time window (time_from / time_to in UTC), coin, optional amount, and memo.

Important detail: if the user enters time without seconds (e.g. 20:38), time_from is set to 20:38:00 and time_to to 20:38:59. This means the search covers exactly one minute. If seconds are specified explicitly, the window equals one second.

TXCandidate

A unified search result. All adapters return a list of objects of this type, so main.py has no knowledge of the internal differences between networks. Contains the transaction hash, timestamp, amount, sender and recipient addresses, memo, and confidence value.

3. Blockchain adapters

All adapters solve the same problem using different methods: retrieve transactions for a given time range, since no public API supports server-side filtering by amount.

3.1 TRON (tron.py)

The most straightforward implementation. TronGrid supports min_block_timestamp and max_block_timestamp parameters in the contract events request, which allows retrieving only Transfer events for the needed range without extra steps.

Endpoint: `GET /v1/contracts/{contract}/events`

Parameters: `event_name=Transfer, min_block_timestamp, max_block_timestamp, limit=200`

Pagination: `fingerprint` — a cursor from the response meta field, passed to the next request.

Notes: TRON USDT uses 6 decimal places (decimals=6), so the amount is stored in sun units and divided by 10^6 . A standard TRC-20 Transfer does not contain a text comment.

3.2 TON (ton.py)

The adapter operates in two modes depending on the selected coin.

Jetton mode (USDT, NOT, USDC)

Endpoint: `GET /v2/jettons/{jetton_id}/transfers`

Parameters: `start_date`, `end_date` (Unix seconds), `limit`, `direction=in`

Pagination: `before_lt` — logical time of the last record, the unique chronology mechanism in TON.

The transfer comment (memo) in TON is a native transaction field stored in `message_content.decoded.comment`. This is a standard blockchain feature, not a workaround.

Native TON mode

Endpoint: `GET https://toncenter.com/api/v3/transactions`

Parameters: `start_utime`, `end_utime` (Unix seconds), `limit`, `offset`

TONAPI does not provide a global endpoint with a time filter for native transactions, so TONCenter v3 is used for this mode.

3.3 EVM networks (eth.py)

Ethereum and compatible networks (BSC, Base) do not support time ranges in the API directly — all requests operate on block numbers. Therefore the adapter performs a preliminary timestamp-to-block conversion step.

Step 1 — conversion: `GET`

`/api?module=block&action=getblocknobytime×tamp=...` — two calls for `time_from` (closest=after) and `time_to` (closest=before).

Step 2 — request: `GET`

`/api?module=account&action=token&contractaddress=...&startblock=X&endblock=Y` — without the address parameter, only the token contract.

Pagination: `page / offset`, up to 1000 records per page.

Multi-chain support is provided through a configuration dictionary: each network has its own API URL and contract addresses. Adding a new EVM network requires two lines of code.

3.4 Bitcoin (btc.py)

The most labour-intensive adapter. The Bitcoin API supports neither time nor amount filtering — only page-by-page block iteration is available.

Step 1 — height estimate: calculated from the current network tip and time difference:
`tip_height = (now - target_ts) / 600`. Bitcoin produces ~1 block every 600 seconds.

Step 2 — find blocks: `GET /api/v1/blocks/{height}` returns 10 blocks at a time. We iterate until blocks are older than `time_from`.

Step 3 — block transactions: `GET /api/block/{hash}/txs/{start_index}` — 25 TX per call. Amount filtering is fully local.

Additionally: the adapter extracts data from `OP_RETURN` — a special field where some services write arbitrary text. This is Bitcoin's equivalent of memo.

4. Scoring algorithm

After retrieving transactions from the API, each candidate is assigned a confidence score from 0 to 1. Candidates below the threshold (0.4 by default) are discarded. The rest are sorted in descending order.

Criterion	Weight	Logic
Time	0.5	Linear from <code>time_from</code> . TX at 20:38:00 → 1.0, TX at 20:38:59 → ~0.0
Amount	0.4	Exact match → 1.0. Within tolerance → linear decay. Outside → rejected immediately.
Memo	0.1	Case-insensitive substring match → 1.0. No match → 0.0.

4.1 Formula

```
confidence = 0.5 × time_score + 0.4 × amount_score + 0.1 × memo_score
```

4.2 Hard amount rejection

If the user specified an amount and a candidate's deviation exceeds the tolerance, the function immediately returns 0.0 without computing other components. This excludes transactions with a mismatched amount regardless of their time score.

4.3 Weight redistribution

If a criterion is not provided by the user, its weight is automatically redistributed:

- No amount and no memo → `confidence = time_score`
- No amount, memo present → `0.9 × time + 0.1 × memo`
- Amount present, no memo → weights 0.5 and 0.4 are normalized proportionally

5. Command-line interface

The utility is implemented as an interactive TUI using the `prompt_toolkit` library. It operates in two screens.

Screen 1 — network selection

ASCII art with the utility name and author credit. Below — a four-network menu navigated with arrow keys `↑ ↓` and confirmed with `Enter`.

Screen 2 — parameter input

Step-by-step input in four stages:

- Transaction time in UTC (required) — format `DD.MM.YYYY HH:MM` or `DD.MM.YYYY HH:MM:SS`
- Coin (required) — selected from the list available for the chosen network
- Amount (optional) — press `Enter` to skip
- Memo / comment (optional) — press `Enter` to skip

After all parameters are entered, a summary is shown and confirmation is requested before the search begins.

Result output

Results are displayed as a table: number, full transaction hash, confidence percentage, amount, and time. When there are more than 100 candidates, the user is prompted to set a limit or export all results to an Excel file.

6. Installation and usage

6.1 Requirements

- Python 3.10+
- `pip` or `pip3`

6.2 Installation

```
git clone <repo> && cd txfetch
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

6.3 Run

```
python3 main.py
```

6.4 Dependencies

- requests >= 2.31 — HTTP requests to all APIs
 - prompt_toolkit >= 3.0 — interactive TUI
 - openpyxl >= 3.1 — Excel export
-

7. Important notes

7.1 Time is always UTC

All adapters work exclusively with UTC. The user must manually convert local time to UTC before entering it. For example, 02:39 UTC+3 corresponds to 23:39 UTC of the previous day.

7.2 API rate limits

Public APIs without a key have rate limits. TronGrid without a key allows ~1 request per second, Etherscan — 5 requests per second. Exceeding limits returns a 429 error. For stable operation, it is recommended to obtain a free API key from the respective service.

7.3 Result accuracy

The utility returns candidates, not a guaranteed answer. When many transfers of the same amount occur within one minute, the list may contain several entries with similar confidence scores. To improve accuracy, providing a memo when known is recommended.